

Document technique

Requêtez une base de données avec SQL

Nadia ZITOUT
09/2025

Sommaire

Introduction.....	3
Etape 1 – Exploration des types de données	4
Etape 2 – Création du schéma relationnel et génération du code SQL.....	5
Etape 3 : Création des tables et chargement des données	6
Etape 4/5 – Analyses SQL.....	10
Conclusion	11

Introduction

Le présent document technique a pour objectif de présenter la conception, la création et le chargement d'une base de données destinée à l'analyse du marché des assurances habitation.

Il regroupe l'ensemble des étapes du projet, à savoir :

- L'exploration des fichiers CSV fournis et la construction d'un dictionnaire des données ;
- La conception du schéma relationnel et la génération du code SQL permettant de créer la base de données ;
- La création des tables dans le SGBD et le chargement des données issues des fichiers sources ;
- La mise en évidence des incohérences détectées et des solutions mises en place.

Des captures d'écran sont intégrées tout au long du document afin d'illustrer les différentes étapes.

Etape 1 – Exploration des types de données

Dans cette première étape, les fichiers CSV fournis **contrat** et **région** (*contrats clients et référentiel extraites du site de data.gouv.fr*) ont été consultés afin de comprendre les données disponibles.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Contrat_ID	No_voie	B_T_Q	Type_de_voie	Voie	Code_dep_c	Code_postal	Surface	Type_local	Occupation	Type_contrat	Formule	Valeur_declaree	Prix_cotisation_mensuel	
2	100601	190 A		RUE	CENTRALE	1350	1370	50	Appartement	Locataire	Residence pi	Integral	0-25000	25	
3	100602	347		RUE	DU CHATEAU	1103	1170	48	Appartement	Locataire	Residence pi	Classique	0-25000	30	
4	100603	58		AV	DU MONT BL	1143	1220	131	Appartement	Propriétaire	Residence pi	Integral	25000-50000	57	
5	100604	140		RUE	DE L'ABBE JO	1288	1630	109	Maison	Locataire	Residence pi	Integral	25000-50000	43	

contrat.csv

	A	B	C	D	E	F	G	H	I
1	Code_dep_c	reg_code	reg_nom	aca_nom	dep_nom	com_nom_maj_court	dep_code	dep_nom_num	
2	1001	84	Auvergne-Rhône-Alpes	Lyon	Ain	LABERGEME	1	Ain (01)	
3	1002	84	Auvergne-Rhône-Alpes	Lyon	Ain	LABERGEME	1	Ain (01)	

region.csv

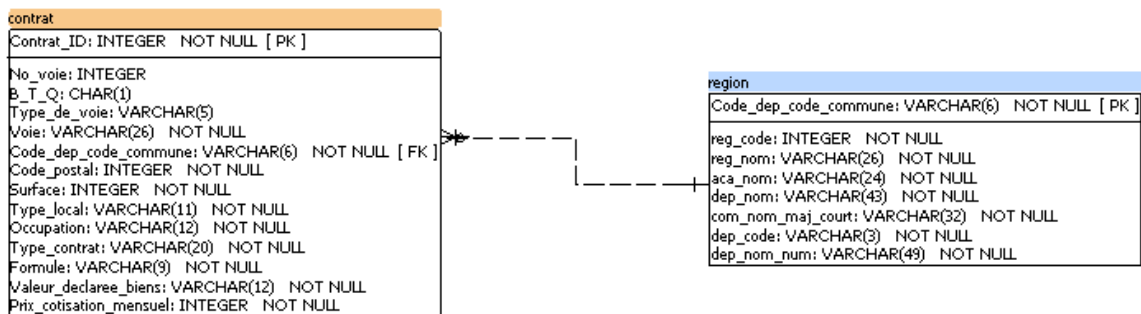
Les fichiers CSV ont été analysés pour identifier les colonnes, les types de données et les contraintes afin de remplir le dictionnaire de données.

	Nom des colonnes	Type de données	Taille	Clé	Description
CONTRAT.CSV	Contrat_ID	INT		Clé primaire	Id unique pour les contrats
	No_voie	INT			Numéro dans la voie pour l'adresse du logement assuré
	B_T_Q	CHAR	1		Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Type_de_voie	VARCHAR	5		Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Voie	VARCHAR	26		Libellé de la voie pour l'adresse du logement assuré
	Code_dep_code_commune	VARCHAR	6	Clé secondaire	Concaténation du code département et code commune pour avoir une clé unique
	Code_postal	INT			Code postal pour l'adresse du logement assuré
	Surface	INT			Surface du logement assuré
	Type_local	VARCHAR	11		Type de logement assuré
	Occupation	VARCHAR	12		Statut d'occupation du logement assuré
	Type_contrat	VARCHAR	20		Type de contrat du logement assuré
	Formule	VARCHAR	9		Formule de l'assurance souscrite
REGION.CSV	Valeur_declaree_biens	VARCHAR	12		Valeur des biens déclarés
	Prix_cotisation_mensuel	INT			Montant mensuel de la cotisation d'assurance
	Code_dep_code_commune	VARCHAR	6	Clé primaire	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INT			Code de la région
	reg_nom	VARCHAR	26		Nom de la région
	aca_nom	VARCHAR	24		Nom de l'académie
	dep_nom	VARCHAR	43		Nom du département
	com_nom_maj_court	VARCHAR	32		Nom court de la commune en majuscule
	dep_code	VARCHAR	3		Code du département
	dep_nom_num	VARCHAR	49		Nom et code du département

Etape 2 – Création du schéma relationnel et génération du code SQL

À partir du dictionnaire des données, un schéma relationnel a été élaboré dans le logiciel **SQL Power Architect** :

- Les tables ont été définies
- Les contraintes de clés primaires et étrangères ont été ajoutées.
- Les colonnes et leurs types de données ont été spécifiés.



Le code SQL suivant a été généré automatiquement par le logiciel à partir du schéma relationnel :

```
CREATE TABLE region (
    Code_dep_code_commune VARCHAR(6) NOT NULL,
    reg_code INT NOT NULL,
    reg_nom VARCHAR(26) NOT NULL,
    aca_nom VARCHAR(24) NOT NULL,
    dep_nom VARCHAR(43) NOT NULL,
    com_nom_maj_court VARCHAR(32) NOT NULL,
    dep_code VARCHAR(3) NOT NULL,
    dep_nom_num VARCHAR(49) NOT NULL,
    PRIMARY KEY (Code_dep_code_commune)
);

CREATE TABLE contrat (
    Contrat_ID INT NOT NULL,
    No_voie INT,
    B_T_Q CHAR(1),
    Type_de_voie VARCHAR(5),
    Voie VARCHAR(26) NOT NULL,
    Code_dep_code_commune VARCHAR(6) NOT NULL,
    Code_postal INT NOT NULL,
    Surface INT NOT NULL,
    Type_local VARCHAR(11) NOT NULL,
    Occupation VARCHAR(12) NOT NULL,
    Type_contrat VARCHAR(20) NOT NULL,
    Formule VARCHAR(9) NOT NULL,
    Valeur_declaree_biens VARCHAR(12) NOT NULL,
    Prix_cotisation_mensuel INT NOT NULL,
    PRIMARY KEY (Contrat_ID)
);

ALTER TABLE contrat ADD CONSTRAINT region_contrat_fk
FOREIGN KEY (Code_dep_code_commune)
REFERENCES region (Code_dep_code_commune)
ON DELETE NO ACTION
ON UPDATE NO ACTION;
```

Etape 3 : Création des tables et chargement des données

Les tables **contrat** et **région** ont été créées dans un SGBD privé à l'aide du code SQL généré à partir du schéma relationnel (étape 2).

```
CREATE TABLE region (  
  Code_dep_code_commune VARCHAR(6) NOT NULL,  
  reg_code INT NOT NULL,  
  reg_nom VARCHAR(26) NOT NULL,  
  aca_nom VARCHAR(24) NOT NULL,  
  dep_nom VARCHAR(43) NOT NULL,  
  com_nom_maj_court VARCHAR(32) NOT NULL,  
  dep_code VARCHAR(3) NOT NULL,  
  dep_nom_num VARCHAR(49) NOT NULL,  
  PRIMARY KEY (Code_dep_code_commune)  
)
```

Requête exécutée avec succès, 0 ligne modifiée. (0.052 s) [Modifier](#)

```
CREATE TABLE contrat (  
  Contrat_ID INT NOT NULL,  
  No_voie INT,  
  B_T_Q CHAR(1),  
  Type_de_voie VARCHAR(5),  
  Voie VARCHAR(26) NOT NULL,  
  Code_dep_code_commune VARCHAR(6) NOT NULL,  
  Code_postal INT NOT NULL,  
  Surface INT NOT NULL,  
  Type_local VARCHAR(11) NOT NULL,  
  Occupation VARCHAR(12) NOT NULL,  
  Type_contrat VARCHAR(20) NOT NULL,  
  Formule VARCHAR(9) NOT NULL,  
  Valeur_declaree_biens VARCHAR(12) NOT NULL,  
  Prix_cotisation_mensuel INT NOT NULL,  
  PRIMARY KEY (Contrat_ID)  
)
```

Requête exécutée avec succès, 0 ligne modifiée. (0.049 s) [Modifier](#)

```
ALTER TABLE contrat ADD CONSTRAINT region_contrat_fk  
FOREIGN KEY (Code_dep_code_commune)  
REFERENCES region (Code_dep_code_commune)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION
```

Tables et vues

Rechercher dans les tables (2)

☐	Table	Moteur?	Interclassement?	Longueur des données?	Longu l'index
☐	contrat	InnoDB	utf8mb4_0900_ai_ci	16,384	
☐	région	InnoDB	utf8mb4_0900_ai_ci	16,384	
	2 au total	InnoDB	utf8mb4_0900_ai_ci	32,768	

Les données issues des fichiers CSV ont été importées, puis vérifiées afin de s'assurer de la correspondance avec les fichiers sources.

Problème rencontré lors du chargement

Lors du chargement des données, une incohérence est apparue lors de l'importation des fichiers contact et région. Cette erreur se produit lorsqu'une valeur insérée ou mise à jour ne correspond pas à une valeur existante dans la table référencée par la clé étrangère.

Cannot add or update a child row: a foreign key constraint fails ('OC_Nadia_P3'. 'contrat', CONSTRAINT 'region_contrat_fk' FOREIGN KEY ('Code_dep_code_commune') REFERENCES 'region' ('Code_dep_code_commune')) 19/11/19 Requête SQL

Une requête SQL a été réalisée afin de détecter les incohérences :

```
SELECT DISTINCT c.Code_dep_code_commune
FROM contrat c
LEFT JOIN region r
ON c.Code_dep_code_commune = r.Code_dep_code_commune
WHERE r.Code_dep_code_commune IS NULL
```

- Les tables **contrat** et **région** sont comparées sur la colonne correspondant au code de département (Code_dep_code_commune pour **contrat** et Code_dep_code_commune pour **région**).
- La requête liste uniquement les codes de contrat qui n'existent pas dans la table des régions.
- L'utilisation de la clause DISTINCT permet de ne lister chaque code qu'une seule fois.
- L'exécution de la requête a permis de détecter trois codes absents dans la table **région**, ce qui empêchait l'application de la contrainte de clé étrangère entre les deux tables.

Code_dep_code_commune
97460
97434
97470

Solution mise en œuvre

La contrainte de clé étrangère a été supprimée pour permettre l'import complet des données. Les données originales n'ont pas été modifiées afin de rester conformes au fichier CSV.

```
CREATE TABLE region (
  Code_dep_code_commune VARCHAR(6) NOT NULL,
  reg_code INT NOT NULL,
  reg_nom VARCHAR(26) NOT NULL,
  aca_nom VARCHAR(24) NOT NULL,
  dep_nom VARCHAR(43) NOT NULL,
  com_nom_maj_court VARCHAR(32) NOT NULL,
  dep_code VARCHAR(3) NOT NULL,
  dep_nom_num VARCHAR(49) NOT NULL,
  PRIMARY KEY (Code_dep_code_commune)
)
```

Requête exécutée avec succès, 0 ligne modifiée. (0.072 s) [Modifier](#)

```
CREATE TABLE contrat (
  Contrat_ID INT NOT NULL,
  No_voie INT,
  B_T_Q CHAR(1),
  Type_de_voie VARCHAR(5),
  Voie VARCHAR(26) NOT NULL,
  Code_dep_code_commune VARCHAR(6) NOT NULL,
  Code_postal INT NOT NULL,
  Surface INT NOT NULL,
  Type_local VARCHAR(11) NOT NULL,
  Occupation VARCHAR(12) NOT NULL,
  Type_contrat VARCHAR(20) NOT NULL,
  Formule VARCHAR(9) NOT NULL,
  Valeur_declaree_biens VARCHAR(12) NOT NULL,
  Prix_cotisation_mensuel INT NOT NULL,
  PRIMARY KEY (Contrat_ID)
)
```

Requête exécutée avec succès, 0 ligne modifiée. (0.045 s) [Modifier](#)

Résultats obtenus

- ✓ Les tables ont été créées avec succès.
- ✓ Toutes les données des fichiers CSV ont été importées.
- ✓ Le nombre de lignes dans les tables est identique à celui des fichiers sources.

Tables et vues

Rechercher dans les tables (2)

☐	Table	Moteur [?]	Interclassement [?]	Longueur des données [?]	Longueur de l'index [?]	Espace inutilisé [?]	Incrément automatique [?]	Lignes [?]	Commentaire [?]
☐	contrat	InnoDB	utf8mb4_0900_ai_ci	16,384	0	0		0	
☐	region	InnoDB	utf8mb4_0900_ai_ci	16,384	0	0		0	
☐	2 au total	InnoDB	utf8mb4_0900_ai_ci	32,768	0	0			

Table **Contrat** :

30,335 lignes ont été importées. 12:56:57 [Requête SQL](#)

[Afficher les données](#) [Afficher la structure](#) [Modifier la table](#) [Nouvel élément](#)

Limite
 Longueur du texte

SELECT * FROM "contrat" LIMIT 50 (0.001 s) [Modifier](#)

☐	Modification	Contrat_ID	No_voie	B_Y_Q	Type_de_voie	Voie	Code_dep_code_commune	Code_postal	Surface
☐	modifier	100601	190	A	RUE	CENTRALE	1350	1370	50
☐	modifier	100602	347	NULL	RUE	DU CHATEAU	1103	1170	48
☐	modifier	100603	58	NULL	AV	DU MONT BLANC	1143	1220	131
☐	modifier	100604	140	NULL	RUE	DE L'ABBE JOLIVET	1288	1630	109
☐	modifier	100605	39	NULL	RUE	BUFFON	1033	1200	109
☐	modifier	100606	8	NULL	RUE	DE GENEVE	1354	1630	53
☐	modifier	100607	2	NULL	RUE	DU RECULET	1354	1630	59
☐	modifier	100608	1403	NULL	RUE	JEAN DE GINGINS	1143	1220	93
☐	modifier	100609	226	NULL	ALL	DES CAPUCINES	1354	1630	117
☐	modifier	100610	276	NULL	RTE	DE POUIGNY	1288	1630	36

Page

Résultat entier ☐ 30,335 lignes

Modification

Sélectionnée(s) (0)

Table **Région** :

38,916 lignes ont été importées. 12:58:43 [Requête SQL](#)

[Afficher les données](#) [Afficher la structure](#) [Modifier la table](#) [Nouvel élément](#)

Limite
 Longueur du texte

SELECT * FROM "region" LIMIT 50 (0.000 s) [Modifier](#)

☐	Modification	Code_dep_code_commune	reg_code	reg_nom	aca_nom	dep_nom	com_nom_maj_court	dep_code	dep
☐	modifier	10001	44	Grand Est	Reims	Aube	L' ABBAYE SOUS PLANCY	10	Aube
☐	modifier	10002	44	Grand Est	Reims	Aube	AILLEVILLE	10	Aube
☐	modifier	10003	44	Grand Est	Reims	Aube	AIX EN OTHE	10	Aube
☐	modifier	10004	44	Grand Est	Reims	Aube	ALLIBAUDIERES	10	Aube
☐	modifier	10005	44	Grand Est	Reims	Aube	AMANCE	10	Aube
☐	modifier	10006	44	Grand Est	Reims	Aube	ARCIS SUR AUBE	10	Aube
☐	modifier	10007	44	Grand Est	Reims	Aube	ARCONVILLE	10	Aube
☐	modifier	10008	44	Grand Est	Reims	Aube	ARGANCON	10	Aube
☐	modifier	10009	44	Grand Est	Reims	Aube	ARRELLES	10	Aube
☐	modifier	1001	84	Auvergne-Rhône-Alpes	Lyon	Ain	L ABERGEMENT CLEMENCIAT	1	Ain (

Page

Résultat entier ☐ 38,916 lignes

Modification

Sélectionnée(s) (0)

Etape 4/5 – Analyses SQL

Dans cette étape, les requêtes SQL demandées ont été réalisées sur les tables **contrat** et **région** importées à l'étape 3. L'objectif était de construire des requêtes et de comprendre les résultats produits. Voici quelques exemples représentatifs de requêtes SQL. *(Le fichier Word associé contient le code SQL complet)*

Requête 4 : Quels sont les 5 contrats qui ont les surfaces les plus élevées ?

```
SELECT  
Contrat ID  
FROM contrat  
ORDER BY Surface DESC LIMIT 0,5
```

Contrat_ID
104211
105463
130878
100822
109872

Requête 9 : Quelle est la surface moyenne des contrats à Paris ?

```
SELECT AVG(Surface)  
FROM contrat c  
JOIN region r ON c.Code dep code commune = r.Code dep code commune  
WHERE r.com nom_maj_court LIKE 'PARIS%'
```

AVG(Surface)
51.7695

Requête 6 : Quel est le nombre de contrats pour chaque catégorie de prix de la valeur déclarée des biens ?

```
SELECT Valeur_declaree_biens AS categorie,  
COUNT(*) AS nbr_contrats  
FROM contrat  
GROUP BY Valeur_declaree_biens
```

categorie	nbr_contrats
0-25000	22720
25000-50000	6815
100000+	104
50000-100000	696

Conclusion

Ce projet a permis de mettre en place une base de données complète à partir de fichiers sources bruts. Les données ont d'abord été explorées et introduit dans un dictionnaire afin de préciser les colonnes, leurs types et leurs contraintes. Cette étape a servi de base à la création d'un schéma relationnel et du code SQL associé.

Lors du chargement dans le SGBD, certaines incohérences ont été identifiées, notamment des Code_dep_code_commune présents dans la table **contrat** mais absents de la table **région**. Une solution adaptée a été appliquée afin de garantir l'intégrité des fichiers d'origine.

Enfin, plusieurs analyses SQL ont été réalisées.